Module 2 – MongoDB (Detailed Notes)

1. Introduction to MongoDB

MongoDB is a NoSQL, document-oriented database designed for high performance, scalability, and flexibility. Unlike traditional relational databases, MongoDB stores data in a JSON-like format called BSON (Binary JSON).

Key Characteristics:

- Schema-less (flexible structure for documents)
- High scalability and performance
- Stores data in collections and documents
- Suitable for modern, data-driven applications

---

2. Collections and Documents

MongoDB organizes data into:

a. Collections

- Equivalent to tables in relational databases
- Do not enforce a strict schema
- Can store documents with different structures

b. Documents

- Equivalent to rows in relational databases
- Stored in JSON-like format (BSON)
- Consist of key-value pairs

Example Document:

```
{
  "name": "Ali",
  "age": 22,
  "course": "MERN"
}
```

Key Advantages:

- Flexibility in data structure
- Easy to modify without affecting existing data
- Better suited for dynamic applications

---

3. CRUD Operations

CRUD stands for Create, Read, Update, and Delete — the four basic operations used to interact with a database.

a. Create (Insert Data)

- **insertOne()** – Inserts a single document

```
db.students.insertOne({ name: "Ali", age: 22 });
```

b. Read (Retrieve Data)

- **find()** – Retrieves multiple documents

```
db.students.find({ age: 22 });
```

- **findOne()** – Retrieves a single document

```
db.students.findOne({ name: "Ali" });
```

c. Update (Modify Data)

- **updateOne()** – Updates a single document

```
db.students.updateOne(
  { name: "Ali" },
  { $set: { age: 23 } }
);
```

d. Delete (Remove Data)

- **deleteOne()** – Deletes a single document

```
db.students.deleteOne({ name: "Ali" });
```

Important Notes:

- Queries use JSON-like syntax.
- Update operations often use operators like `$set`, `$inc`.

---

4. Indexes

Indexes are used to improve the performance of queries by allowing faster data retrieval.

Why Indexes are Important:

- Reduce the amount of data scanned during queries
- Improve speed of search operations
- Essential for large datasets

Example:

```
db.students.createIndex({ name: 1 });
```

Types of Indexes:

- Single field index
- Compound index
- Text index
- Unique index

Considerations:

- Too many indexes can slow down write operations
- Use indexes strategically based on query patterns

---

5. Aggregation Pipelines

Aggregation pipelines are used to process and transform data for analytics and reporting.

How it Works:

- Data passes through multiple stages
- Each stage performs a specific operation

Common Stages:

$match

- Filters documents (similar to WHERE clause)

```
{ $match: { age: 22 } }
```

$group

- Groups data and performs calculations

```
{ $group: { _id: "$age", count: { $sum: 1 } } }
```

$sort

- Sorts documents

```
{ $sort: { age: -1 } }
```

$project

- Selects specific fields

```
{ $project: { name: 1, age: 1 } }
```

Example Pipeline:

```
db.students.aggregate([
  { $match: { age: 22 } },
  { $group: { _id: "$age", total: { $sum: 1 } } }
]);
```

Benefits:

- Powerful data transformation
- Useful for reports and analytics
- Reduces need for complex application logic

---

6. Relationships in MongoDB

MongoDB handles relationships differently from relational databases.

a. Embedding (Nested Documents)

- Stores related data inside a single document

Example:

```
{
  "name": "Ali",
  "courses": [
    { "title": "MERN", "duration": "3 months" }
  ]
}
```

Advantages:

- Faster read operations
- Fewer queries required

Disadvantages:

- Data duplication
- Limited scalability for large nested data

---

b. Referencing (Linked Documents)

- Stores references (IDs) to other documents

Example:

```
{
  "name": "Ali",
  "courseId": "12345"
}
```

Advantages:

- Reduces data duplication
- Better for complex relationships

Disadvantages:

- Requires additional queries (joins-like behavior)

---

7. Security in MongoDB

Securing the database is essential in production environments.

Best Practices:

- Enable authentication (username and password)
- Use role-based access control (RBAC)
- Restrict network access (IP whitelisting)
- Encrypt data (in transit and at rest)
- Regularly update MongoDB version

---

8. Key Concepts to Remember

- MongoDB stores data as JSON-like documents (BSON format).
- Collections contain documents without strict schema enforcement.
- CRUD operations are fundamental for interacting with data.
- Indexes significantly improve query performance.
- Aggregation pipelines provide powerful data processing capabilities.
- Relationships can be handled using embedding or referencing.
- Proper security measures must always be implemented.

---

9. Summary

MongoDB is a flexible and scalable NoSQL database ideal for modern applications. Its document-based structure, powerful querying capabilities, and aggregation framework make it a strong choice for handling large and dynamic datasets. Understanding CRUD operations, indexing, aggregation, and relationships is essential for effective database design and performance optimization.